Classification of Legal Text

UE24CS352A Machine Learning mini-project / Team 9 / Problem 5
Sai Sathvik Gullipalli (PES2UG24CS430)
Priyanka Mallikarjuna Reddy (PES2UG24CS378)

Problem statement

Classify an English US Supreme Court opinion into its broad legal issue area. We compare lexical, topic, and document-embedding representations and demonstrate similar-opinion retrieval. This project adapts Iyer's Stanford study [1], with an independent dataset cleaning and evaluation procedure.

Dataset and preparation

The original textacy release [2] contains **8,419 opinions**, spanning 1946-11-18 to 2016-06-27. The official mapping [3] has 14 named issue areas. After cleaning, this run retains **8,373 opinions in 13 supported categories**, across 8,373 case groups.

We exclude empty text and missing/unknown labels, remove identical normalized text, and join records through shared citations or text hashes. Groups with conflicting labels are excluded. Categories require at least five independent case groups. Overall, 46 records are excluded, including 17 records in conflicting groups and 3 duplicate copies. Rare-category exclusions: Private Action (1 case group).

Partition	Opinions	Case groups
Train	5,023	5,023
Validation	1,675	1,675
Test	1,675	1,675

A fixed, approximately 60/20/20 grouped, stratified split uses seed 42. Case groups never cross partitions. Every vocabulary, topic model, and embedding learns from training opinions only. Validation chooses configurations. Test opinions evaluate the frozen models once.

Approach and implementation

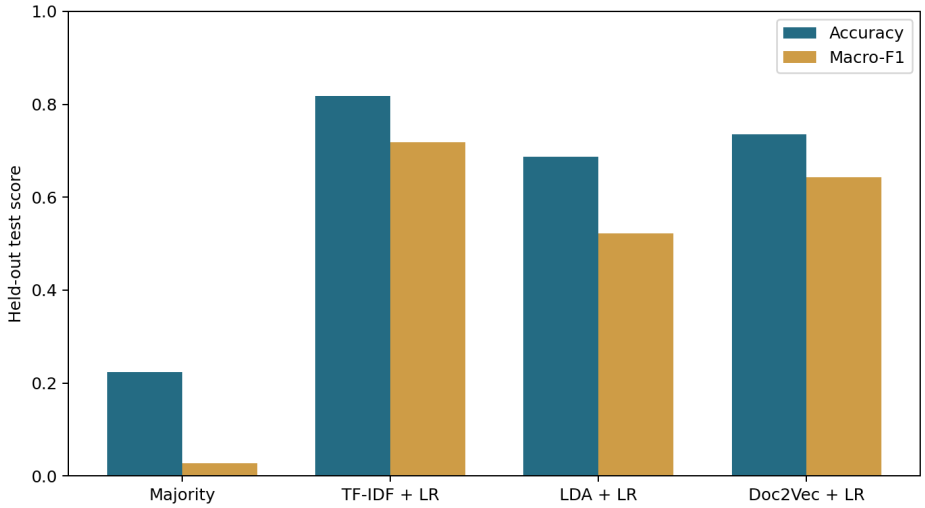
TF-IDF + LR: sparse unigram/bigram features, sublinear term frequency, English stop words, and at most 30,000 features.
LDA + LR: count features with at most 15,000 words and 20 or 40 topics. Online LDA uses five passes. Counts replace the reference's stated TF-IDF input because LDA models word counts.
Doc2Vec + LR: 100-dimensional PV-DBOW embeddings, 30 epochs, one worker. All classification partitions use inferred vectors, with stable text-specific seeds.

Each Logistic Regression compares C = 0.1, 1, 10. Validation macro-F1 chooses the configuration, with accuracy then inference latency as tie-breaks. Python modules implement training and inference. Separate Jupyter/Colab notebooks provide experiments and a saved-model demo. Exported artifacts preserve provenance, partitions, and metrics.

Results and conclusions

Frozen test comparison / 1,675 held-out opinions

Model	Accuracy	Macro-F1	Weighted F1
Majority baseline	0.224	0.028	0.082
TF-IDF + LR	0.819	0.719	0.814
LDA + LR	0.688	0.522	0.677
Doc2Vec + LR	0.735	0.643	0.730



Validation selects **TF-IDF + LR** with macro-F1 0.682. Its held-out accuracy is 0.819 and macro-F1 is 0.719. The majority baseline reaches accuracy 0.224. Macro-F1 gives each category equal weight, revealing performance differences that accuracy can conceal in an imbalanced corpus.

Measured per-opinion inference on the execution CPU: TF-IDF + LR: 7.1 ms, LDA + LR: 5.1 ms, Doc2Vec + LR: 105.1 ms. These timings include feature transformation and classification and depend on hardware and opinion length.

Demonstration and limitations

The demo loads saved models, displays a category and top-three uncalibrated probabilities, and shows influential terms for TF-IDF. Doc2Vec cosine similarity retrieves five distinct training-case citations with metadata and excerpts. This retrieval is qualitative: textual resemblance does not establish binding authority or a valid legal argument.

Results apply to historical US Supreme Court opinions. Random grouped splits do not measure future-case generalization. Imbalanced categories, possible source metadata mismatches, and boilerplate affect reliability. Gensim caps each Doc2Vec pass at 10,000 retained tokens, which can omit later context in long opinions. Short user passages differ from full training opinions. No claim of generalization to Indian law is made. The reference paper's prose and results table conflict, so its scores are not reused as our results.

References and reproducibility

[1] K. Iyer. *Classification of Legal Text*, Stanford CS229, 2020. cs229.stanford.edu/proj2020spr/report/Iyer.pdf
[2] textacy-data. Supreme Court corpus, Python 3 release v1.0. github.com/bdewilde/textacy-data, supreme_court_py3_v1.0
[3] textacy SupremeCourt mapping. Metadata: Spaeth, Epstein et al., Supreme Court Database, 2016 Release 1. CC BY-NC 3.0 US attribution, per corpus maintainer.

Dataset SHA256: 2909fc637b44326b7f588371828f6446883dc9c66cf31f6a5805a50248fbb0f4
Reproduction: README, pinned requirements, saved split manifest, validation candidates, test predictions, and per-category reports in the private repository.